

- Graph Traversal
 - DFS
 - BFS
 - Articulation Point关节点
 - 定义：关节点
 - Tarjan 算法求解割点（关节点）思路梳理
 - 1. 核心状态维护 (两个关键数组)
 - 2. DFS 递归转移流程
 - 3. 割点（关节点）判断法则
- Strong Connected Components
 - 一、什么是强连通分量 (SCC)?
 - 二、求解 SCC 的两大经典算法
 - 1. Kosaraju 算法 (两次 DFS 法)
 - 2. Tarjan 算法 (单次 DFS + 栈)
 - 三、Tarjan 算法求 SCC 的核心思路
 - 1. 状态维护
 - 2. DFS 转移流程
 - 3. SCC 判定法则 (出栈结算)
 - 所有的树天生是二部图
 - 图的直径
 - 两次 DFS / BFS 法 (最直观)
 - 证明过程（反证法）
 - 情况一：路径与直径相交
 - 情况二：路径与直径不相交
- 拓扑排序
 - 一、什么是拓扑排序?
 - 二、拓扑排序的核心前提
 - 三、如何求解? (Kahn 算法)
- 最小生成树MST
 - 一、Kruskal 算法 (克鲁斯卡尔): 以“边”为核心的全局贪心
 - 1. 核心数据结构: 并查集 (Disjoint Set Union, DSU)
 - 2. 复杂度与适用场景
 - 二、Prim 算法 (普林): 以“点”为核心的局部扩张
 - 1. 核心算法流程
 - 2. 核心数据结构: 优先队列 (Min-Heap)
 - 3. 复杂度与适用场景
 - 直观对比

- SSSP
 - 一、SSSP 问题的核心动作：“松弛 (Relaxation)”
 - 二、SSSP 的四大经典解法工具箱
 - 1. 无权图 / 所有边权相等：BFS (广度优先搜索)
 - 2. 有向无环图 (DAG)：拓扑排序 + 动态规划
 - 3. 非负权图：Greedy的Dijkstra 算法 (迪杰斯特拉)
 - 4. 包含负权边的一般图：DP的Bellman-Ford 算法 (贝尔曼-福特)
- 全源最短路径问题 (All-Pairs Shortest Paths, APSP)
- 1. 核心算法对比表
- 2. 算法详细拆解
 - A. Floyd-Warshall 算法 (弗洛伊德算法)
 - B. Johnson 算法
 - C. 多源 Dijkstra 算法
- 3. 算法选型决策流程 (笔记建议)

Graph Traversal

主要分为BFS和DFS两种基本方法。对一个图用BFS和DFS会张成一个树。

DFS

1778025856259

用Stack栈实现。DFS是LIFO的后进先出的结构

1. 顺序为 $v1 \rightarrow v3 \rightarrow v5 \rightarrow v6 \rightarrow v4 \rightarrow v7$, 构建单边树
2. 开始弹栈。体现为Back edge回边。例如寻找v7的其他连接节点v6. 没有其他节点则弹出，反复执行直到栈中只剩v1
3. 1778026109610

BFS

用队列来实现BFS，BFS是FIFO的先进先出结构

1. 顺序为 $v1 \rightarrow v3 \rightarrow v3 \rightarrow v5 \rightarrow v4 \rightarrow v7 \rightarrow v6$
2. 从v1开始，寻找其他连接，体现为回边back edge
3. 1778026250703

Articulation Point关节点

定义：关节点

删去某节点后，一张图的连通受到变化，则称该节点为关节点，也就是割点。

Tarjan 算法求解割点（关节点）思路梳理

算法的核心思想是通过深度优先搜索（DFS）给图生成一棵 DFS 遍历树，并利用时间戳和回溯值来判断某个节点是否是图中不可或缺的“桥梁”。

1. 核心状态维护 (两个关键数组)

在遍历过程中，对每个顶点 u 记录两个至关重要的时间戳：

1. $disc[u]$ (Discovery Time - 发现时间)：节点 u 在 DFS 过程中第一次被访问到的顺序编号（递增）。
2. $low[u]$ (Lowest Time - 最早回溯时间)：节点 u 及其子树中的节点，通过最多一条回边（非树边），能够到达的、最早被发现的祖先节点的 $disc$ 值。

2.DFS 递归转移流程

初始化时，对当前节点 u ，设置 $disc[u] = low[u] = ++time$ 。

随后遍历 u 的每一个相邻节点 v ：

- 情况 A：如果 v 未被访问过（这是一条“树边”）
 1. 记录父子关系：设置 $parent[v] = u$ 。
 2. 增加子节点计数（用于后续判断根节点）。
 3. 向下递归：调用 $dfs(v)$ 。
 4. 向上回溯并更新：当 v 的 DFS 结束返回时，说明 v 及其子树的 low 值已计算完毕。此时更新 u 的回溯值： $low[u] = \min(low[u], low[v])$
 5. 判断割点：根据下方的“判断法则”检查 u 是否为割点。
- 情况 B：如果 v 已被访问过，且 v 不是 u 的父节点（这是一条“回边”）
 - 说明找到了一条绕过父节点回到祖先的路径。此时只需更新 u 的回溯值（不递归）： $low[u] = \min(low[u], disc[v])$

3. 割点（关节点）判断法则

在处理 情况 A（从子节点 v 回溯到 u ）时，通过以下两条规则判断 u 是否为割点：

- 法则一：针对 DFS 树的“根节点”

- 条件： $\text{parent}[u] == -1$ 且 拥有超过 1 个子节点（即遍历出两棵及以上的独立子树）。
- 原理解释：因为这是整棵树的起点，如果它有多个子树，说明这些子树之间没有其他路径相连（否则它们会在 DFS 过程中连通，不会成为根节点独立的子树）。移除根节点，这些子树就会彻底断开。

- 法则二：针对 DFS 树的“非根节点”

- 条件： $\text{parent}[u] \neq -1$ 且 $\text{low}[v] \geq \text{disc}[u]$ 。
- 原理解释： $\text{low}[v]$ 是子节点 v 及其子树能往回跑到的最高点。如果 $\text{low}[v] \geq \text{disc}[u]$ ，说明 v 的子树无论怎么绕，最早也只能回到 u 本身，根本找不到回到 u 之前祖先节点的路径。此时， u 就是卡住 v 子树的唯一咽喉，移除 u ， v 所在的子树就会和图的其余部分断开连接。

Strong Connected Components

一、什么是强连通分量 (SCC)?

在有向图中，定义如下：

- 强连通 (Strongly Connected)：如果对于有向图中的两个顶点 u 和 v ，既存在从 u 到 v 的路径，也存在从 v 到 u 的路径，则称 u 和 v 是强连通的。
- 强连通图：如果图中的任意两个顶点都是强连通的，这个图就是强连通图。
- 强连通分量 (SCC)：有向图的极大强连通子图。“极大”意味着在这个子图里，你不能再加入任何一个外部节点，否则它就不满足强连通的性质了。

直观理解：SCC 就是有向图里的“内循环圈”。在一个 SCC 内部，你可以从任何一个节点出发，顺着箭头走到该 SCC 内的任何其他节点。

二、求解 SCC 的两大经典算法

求解 SCC 主要有两种线性时间复杂度 $O(V + E)$ 的经典算法。

1. Kosaraju 算法 (两次 DFS 法)

这个算法的数学直觉非常优雅，它依赖于图的转置（将所有有向边反向）：

1. **第一遍 DFS (原图)**：对原图进行后序遍历，记录每个节点的“完成时间”（即该节点及其所有子孙都遍历完退出的时间）。将其压入一个栈中。
2. **图的转置**：把图中所有的边方向反转。
3. **第二遍 DFS (反向图)**：按照栈顶到栈底的顺序（即完成时间从晚到早），在反向图上不断发起 DFS。每一次从栈顶节点出发能够遍历到的所有节点，就构成一个 SCC。

2. Tarjan 算法 (单次 DFS + 栈)

这是 Tarjan 教授的另一个杰作，只需要一次 DFS 就能找出所有 SCC。它不仅代码结构极其精简，而且由于只需要遍历一次，实际运行常数通常比 Kosaraju 小。

三、Tarjan 算法求 SCC 的核心思路

与求割点不同，求 SCC 的 Tarjan 算法引入了一个栈 (Stack) 来保存当前搜索路径上的节点。

1. 状态维护

- **disc[u]**：发现时间。
- **low[u]**：节点 u 或其子树能够追溯到的、且仍在栈中的最早祖先的 **disc** 值。
- **栈 (Stack)**：存储当前尚未归属于某个确定 SCC 的节点。
- **in_stack[u]**：布尔数组，快速判断节点 u 是否在栈中。

2. DFS 转移流程

对未访问的节点 u 发起 DFS：

1. 初始化 **disc[u] = low[u] = ++time**，将 u 压入栈，标记 **in_stack[u] = true**。
2. 遍历 u 的每一个有向邻居 v ：
 - **情况 A**： v 未被访问 (树边) 向下递归调用 **dfs(v)**。回溯时更新：**low[u] = min(low[u], low[v])**。
 - **情况 B**： v 已被访问，且 v 在栈中 (回边 或 跨边) ** 说明找到了一个环路 (或者通向之前环路的捷径)，更新：**low[u] = min(low[u], disc[v])**。(注意：如果 v 已访问但 **不在栈中**，说明 v 属于一个已经处理

完毕并出栈的 SCC，不能用它来更新当前节点的 **low** 值，直接忽略。这是有向图求 SCC 和无向图求割点最大的区别之一。)

3. SCC 判定法则 (出栈结算)

当遍历完 u 的所有邻居，准备结束 u 的 DFS 递归时，进行判定：

- 法则：如果 **$\text{low}[u] == \text{disc}[u]$**
 - 原理解释：这意味着 u 及其子树中的节点最多只能回溯到 u 本身，无法到达 u 的更早的祖先。因此， u 就是当前被发现的这个 SCC 的“根节点”（或称入口节点）。
 - 操作：不断从栈顶弹出节点，直到把 u 本身也弹出来。这批被弹出的节点（包括 u ）共同构成一个完整的 SCC。将它们的 **in_stack** 标记设为 **false**。

所有的树天生是二部图

把树按层数的奇偶性用不同颜色标注，得到的两部分构成一个二部图。

图的直径

两次 DFS / BFS 法 (最直观)

这是最常用、最容易理解和实现的方法。它的核心思想基于一个重要的几何性质。

算法步骤：

1. **第一步**：在树中任选一个节点 u 作为起点，进行一次 DFS 或 BFS，找到距离 u 最远的节点，记为 v 。
2. **第二步**：以刚刚找到的节点 v 为新的起点，再进行一次 DFS 或 BFS，找到距离 v 最远的节点，记为 w 。
3. **结论**：节点 v 到节点 w 的路径就是树的直径，这两点就是直径的两个端点。

为什么它能成立？（原理简述）

这个算法的正确性可以通过反证法证明。简单来说，无论 you 从树里的哪个点出发，顺着边往最远的地方走，最终必然会走到树的某个“极度边缘”的叶子节点上。而这个“最边缘”的节点，必定是整棵树最长路径（直径）的其中一个端点。既然找到了一个端点，从它出发再找最远的点，自然就拉出了整条直径。

注意：此方法仅适用于边权非负的树。如果存在负权边，贪心地找最远点可能会导致错误，此时必须使用树形DP。

当然可以。这个算法的正确性可以通过严谨的数学证明（通常使用反证法）来得出。

为了证明清晰，我们先明确一些数学符号和前提：

- **前提**：给定一棵边权非负的树 T 。
- **距离**：设 $d(x, y)$ 为树中节点 x 到节点 y 的唯一最短路径长度。
- **定理**：对于树中任意一点 u ，距离 u 最远的节点 v ，必定是树的某条直径的端点。

只要证明了这个定理，第一步找到直径的一端 v ，第二步再找离 v 最远的点 w ，根据直径的定义， $d(v, w)$ 自然就是整棵树的最长路径了。

下面我们来证明这个核心定理。

证明过程（反证法）

假设：节点 v 是距离起点 u 最远的点（即对于树中任意点 x ，都有 $d(u, v) \geq d(u, x)$ ），但 v 不是任何一条直径的端点。

设树的真实直径为其端点分别为 s 和 t 的路径，长度为 $d(s, t)$ 。因为这是树的最长路径，所以对于树中任意两点 a, b ，必然有 $d(s, t) \geq d(a, b)$ 。

现在，我们观察起点 u 到最远点 v 的路径 $P(u, v)$ ，与直径路径 $P(s, t)$ 的空间关系。在树这种拓扑结构中，这两条路径只有两种可能的关系：**相交** 或 **不相交**。

情况一：路径 $P(u, v)$ 与 直径 $P(s, t)$ 相交

假设这两条路径相交于节点 c 。

1. 根据我们的已知条件， v 是离 u 最远的点，因此 u 到 v 的距离，一定大于等于 u 到 s 的距离：

$$d(u, v) \geq d(u, s)$$

2. 因为这两条路径交于点 c ，我们可以把路径拆开：

$$d(u, c) + d(c, v) \geq d(u, c) + d(c, s)$$

3. 两边同时消去 $d(u, c)$ ，得到：

$$d(c, v) \geq d(c, s)$$

4. 现在，我们来构造一条新的路径：从 v 走到 c ，再从 c 走到 t 。这条新路径的长度为：

$$d(v, t) = d(v, c) + d(c, t)$$

5. 将第 3 步的结论代入，可以得到：

$$d(v, t) \geq d(s, c) + d(c, t)$$

6. 而 $d(s, c) + d(c, t)$ 正好就是直径的长度 $d(s, t)$ ！所以：

$$d(v, t) \geq d(s, t)$$

7. 但是，前提中我们说过 $d(s, t)$ 是整棵树的直径（最长路径），所以不可能有比它更长的路径，即只能是 $d(v, t) = d(s, t)$ 。

结论 1：这说明 $P(v, t)$ 也是一条直径，这就意味着 v 确实是一条直径的端点。这与我们的假设矛盾！

情况二：路径 $P(u, v)$ 与 直径 $P(s, t)$ 不相交

既然树是连通图，且这两条路径不相交，那么必然存在一条唯一的“桥梁”路径将它们连接起来。

设这条连接路径的两个端点分别为 x 和 y （其中 x 在 $P(u, v)$ 上， y 在 $P(s, t)$ 上）。设这条“桥梁”的长度为 $d(x, y)$ ，因为不相交，所以 $d(x, y) > 0$ 。

1. 同样的已知条件， v 是离 u 最远的点，所以 u 到 v 的距离一定大于等于 u 到 s 的距离：

$$d(u, v) \geq d(u, s)$$

2. 根据节点位置，我们把路径完全展开：

$$d(u, x) + d(x, v) \geq d(u, x) + d(x, y) + d(y, s)$$

3. 两边同时消去 $d(u, x)$ ，得到：

$$d(x, v) \geq d(x, y) + d(y, s)$$

4. 现在，我们来构造一条从 v 到 t 的路径：

$$d(v, t) = d(v, x) + d(x, y) + d(y, t)$$

5. 将第 3 步中 $d(x, v)$ 的不等式代入：

$$d(v, t) \geq [d(x, y) + d(y, s)] + d(x, y) + d(y, t)$$

$$d(v, t) \geq 2 \cdot d(x, y) + d(y, s) + d(y, t)$$

6. 我们知道 $d(y, s) + d(y, t)$ 正是直径的长度 $d(s, t)$ ，代入得：

$$d(v, t) \geq 2 \cdot d(x, y) + d(s, t)$$

7. 因为 $d(x, y) > 0$ ，所以：

$$d(v, t) > d(s, t)$$

结论 2： 我们竟然推出了一条长度严格大于直径 $d(s, t)$ 的路径 $P(v, t)$ ！这违背了“直径是最长路径”的基本定义。这是一个 **绝对的逻辑矛盾**。

无论是情况一还是情况二，只要假设“距离起点的最远点 v 不是直径端点”，都会推导出与已知公理相矛盾的结果。

因此，假设不成立。原命题得证：**从树中任意节点出发，找到的距离它最远的节点，必定是这棵树某条直径的端点。** 两次搜索法在数学逻辑上是严密且无懈可击的。

拓扑排序

拓扑排序 (Topological Sorting) 是图论中一个非常实用且基础的概念，它主要用于解决依赖关系的调度和先后顺序问题。

一、什么是拓扑排序？

简单来说，给定一个 **有向无环图 (DAG, Directed Acyclic Graph)**，拓扑排序就是将图中的所有节点排成一个线性的序列，并且满足一个核心规则：

如果图中存在一条从节点 u 指向节点 v 的边 (即 $u \rightarrow v$)，那么在这个线性序列里， u

**** 必须排在 v 的前面。 ****

生活中的直观例子：

- **大学选课：** 你必须先学完《高等数学》(节点 u)，才能去修《概率论》(节点 v)。高数是指向概率论的先修课。拓扑排序就是为你生成一份不会出现“未满足先修条

件”的四年选课计划表。

- **日常穿衣**：必须先穿袜子再穿鞋，先穿内衣再穿外套。
- **工程编译**：在软件开发中，模块 B 依赖模块 A，编译器必须先编译 A 再编译 B（例如 Makefile 或 Webpack 的依赖解析）。

二、拓扑排序的核心前提

进行拓扑排序的图 **必须是有向无环图 (DAG)**。

- **有向**：依赖关系是有方向的（A 决定 B）。
- **无环**：绝对不能存在环路。如果有环（比如 A 依赖 B，B 依赖 C，C 又依赖 A），这就变成了“先有鸡还是先有蛋”的死循环，任何一个任务都无法开始。因此，**拓扑排序也常被用来检测一个有向图中是否存在环**。

(注：一个图的拓扑排序结果通常不是唯一的，只要满足所有依赖规则即可。)

三、如何求解？（Kahn 算法）

求解拓扑排序最经典、最符合人类直觉的算法是 **Kahn 算法（基于入度）**。它的核心思想就是“不断寻找没有前置依赖的任务”。

我们需要引入一个概念：**入度 (In-degree)** —— 指向某个节点的边的数量。入度为 0，意味着这个节点没有任何前置依赖，可以立刻执行。

Kahn 算法的执行步骤：

1. **统计入度**：计算图中所有节点的初始入度。
 2. **初始化队列**：将所有入度为 0 的节点放入一个队列（或集合）中。
 3. **循环拆解**：
 - 从队列中取出一个节点，将其加入到最终的**拓扑排序结果序列**中。
 - 把它在图中的“存在感”抹除：也就是把从它出发、指向其邻居的边全部删除。
 - 这会导致它所有邻居节点的 **入度减 1**。
 - 如果某个邻居节点的入度减到了 0（说明它的所有前置任务都完成了），就把这个邻居加入队列。
 1. **结束判定**：重复步骤 3 直到队列为空。如果最终结果序列中的节点数等于图的总节点数，说明排序成功；如果小于总节点数，说明图中存在环。
-

最小生成树MST

最小生成树（Minimum Spanning Tree, **MST**）是图论中非常经典且具有极高工程应用价值的问题。比如在电路板布线、通信网络规划、甚至是你在研究的机器人路径网络设计中，要在保证所有节点连通的前提下，使得铺设的“总代价”（边权之和）最小，就需要用到它。

在一个带有权重的无向连通图中，求解 MST 最著名的两种算法是 **Kruskal**（克鲁斯卡尔）算法和 **Prim**（普林）算法。它们都运用了贪心思想（Greedy），但切入点截然不同：一个聚焦于“边”，另一个聚焦于“点”。

一、Kruskal 算法（克鲁斯卡尔）：以“边”为核心的全局贪心

Kruskal 算法的直觉非常质朴：既然要总权重最小，那我就把所有的边按价格从小到大排个序，然后从最便宜的边开始挑。只要挑进来的边不和已经选好的边形成“闭环”（因为树不能有环），我就买下它，直到所有顶点都被连通（即挑够 $V - 1$ 条边）。

1. 核心数据结构：并查集 (Disjoint Set Union, DSU)

我们在挑边的过程中，最大的难题是如何快速判断“加上这条边会不会形成环？”

这就是并查集大显身手的时候：

- 将图中的每个节点看作一个独立的集合。
- 当我们考察边 $u-v$ 时，查询 u 和 v 所在的集合的根节点（Find 操作）。
- 如果根节点相同，说明它们已经在同一个连通块里了，连上这条边必然成环，丢弃。
- 如果根节点不同，说明安全，将这条边加入 MST，并将这两个集合合并（Union 操作）。

2. 复杂度与适用场景

- 时间复杂度：主要是边排序的开销 $O(E \log E)$ ，并查集的查询和合并近乎常数时间 $O(E\alpha(V))$ 。
- 空间复杂度： $O(E + V)$

- **适用场景**：由于它一开始就要对所有边排序，因此特别适合边数较少的 **稀疏图**。
 - **Kruskal is greedy and optimal**
 - **存在性**：连通图中一定有子连通图，子连通图一定有最小的
 - **唯一性**：MST本身不唯一，但是长度是唯一的
 - **最优性**：设Kruskal生成的MST是K，另一个存在的最优解MST是T，K不等于T，欲证明 $|K| = |T|$ 。
 - 按照权重排序，必然存在边e，使得e属于K但是不属于T。其中e是最小的
 - 将e加入到T的图中，必然构成环。(树加边必成环)
 - 再找一条边f，其属于T不属于K，则必有e小于等于f。(因为e属于K，K是对边贪婪的)
 - 因此 $w(f) \geq w(e)$
 - 又因为MST，故知只能取等。
 - 对每条相异的边执行上述过程，即得到 $|K| = |T|$
-

二、Prim 算法（普林）：以“点”为核心的局部扩张

如果说 Kruskal 是上帝视角挑便宜货，那 Prim 就是以某个初始节点为大本营，像墨水滴在纸上一样，**向外不断吞噬距离当前领地最近的新节点**。

1. 核心算法流程

1. 任选一个节点作为起点，将其标记为“已占领集合”（MST 集合）。
2. 观察从“已占领集合”向“未占领集合”伸出的所有边。
3. 贪心地挑出其中**权重最小**的一条边，将这条边及其连接的新节点拉入“已占领集合”。
4. 重复这个过程，直到所有节点都被占领。

2. 核心数据结构：优先队列 (Min-Heap)

为了快速找到“跨越两个集合的最短边”，Prim 算法通常结合最小堆（优先队列）来实现。这让它的代码结构看起来和求解单源最短路径的 Dijkstra 算法极其相似。

3. 复杂度与适用场景

- **时间复杂度**：使用二叉堆优化后通常为 $O(E \log V)$ 。
- **空间复杂度**： $O(V + E)$

- **适用场景**：因为它是在不断更新节点向外的最短距离，不太受边数众多的拖累，因此非常适合边数密集的 **稠密图**（如果是完全图，不用堆优化的基础 $O(V^2)$ 实现反而更快）。
 - **Kruskal** is **greedy** and **optimal**
 - 最优性证明：
 - 必然能找到一条边 e ，其顶点为 u, v 。其中 u 属于 P 不属于 T ， v 不属于 P 属于 T
 - 证明思想类似，此处略
-

直观对比

简而言之：

- **Kruskal**：在整个图里到处捡最便宜的树枝，最后拼成一棵树。
 - **Prim**：种下一颗种子，让它顺着阻力最小的方向慢慢长成一棵树。
-

SSSP

在图论的宏大版图中，如果说最小生成树（MST）解决的是“如何用最少的成本连通所有节点”，那么 **SSSP（Single-Source Shortest Path，单源最短路径）** 解决的就是：“站在地图上的某一点，如何找到前往其他所有节点的最短路线？”

这是日常生活中应用最广的图论算法，从高德/谷歌地图的导航，到互联网路由器中 OSPF（开放最短路径优先）协议的数据包转发，底层核心都是 SSSP。

一、SSSP 问题的核心动作：“松弛 (Relaxation)”

无论是哪种 SSSP 算法，它们底层都在反复做一个极其基础的动作——**松弛操作 (Relaxation)**。

假设我们用一个数组 $dist[v]$ 来记录从源点 S 到节点 v 的已知最短距离（初始时除了 $dist[S] = 0$ ，其余全为 ∞ ）。

当我们考察一条从 u 指向 v 、权重为 w 的边时，我们会做一个灵魂拷问：

“如果我先走到 u ，再通过这条边走到 v ，会不会比我现在已知的走到 v 的路径更近？”

用数学公式表达就是：

如果 $dist[u] + w < dist[v]$ ，那么更新 $dist[v] = dist[u] + w$ 。

这就是“松弛”。所有的 SSSP 算法，本质上只是以不同的顺序和策略去执行这个松弛动作。

二、SSSP 的四大经典解法工具箱

针对不同性质的图，我们有不同的“特效药”算法。按图的约束条件从严到宽，主要有以下四种：

1. 无权图 / 所有边权相等：BFS (广度优先搜索)

- **核心思想**：像水波纹一样一层一层向外扩展。先找到距离为 1 的点，再通过它们找到距离为 2 的点。
- **适用条件**：所有边的权重必须一样（比如迷宫问题）。
- **时间复杂度**： $O(V + E)$ 。

2. 有向无环图 (DAG)：拓扑排序 + 动态规划

你刚刚了解过拓扑排序，它在这里有绝妙的应用！

- **核心思想**：既然图中没有环，时间流逝是单向的。我们直接对图进行 **拓扑排序**，然后按照拓扑序列的顺序，依次对每个节点的所有出边执行“松弛”操作。
 - **优势**：因为前面的节点绝不会被后面的节点影响，所以每个节点只需要处理一次，效率极高，甚至 **允许存在负权边**。
 - **时间复杂度**： $O(V + E)$ 。
-

3. 非负权图：Greedy的Dijkstra 算法 (迪杰斯特拉)

这是 SSSP 中最著名的算法。它的思路和求 MST 的 Prim 算法极其相似（都是贪心 + 优先队列）。

- **核心思想**：维护一个“已确定最短路径的节点集合”。每次从剩余节点中，挑出目前距离源点 S 最近的那个节点 u ，把它加入集合。然后以 u 为跳板，松弛它所有的邻居。

- 这是因为，在**非负权图**中，最小性是唯一的，即不能用“从更长的当前路径再过一个负路径”达到更短的效果。aka”步步贪心”
- **致命弱点：** **不能有负权边**。Dijkstra 的贪心前提是“只要我选了当前最近的点，以后不可能有其他绕远路的走法比它更近”。如果有负权边（越走越近），这个前提就被打破了。
- **时间复杂度：** 使用最小堆优化后为 $O(E \log V)$ 。

堆操作次数：

- 最多进行 $|E||E|$ 次 **push**（每次成功松弛产生一次压堆）。
- 最多进行 $|V||V|$ 次有效的 **pop**（每个节点真正确定最短距离后弹出一次），再加上可能弹出的若干无效旧条目，总弹出次数 $\leq |E| + |V||E| + |V|$ 。
- 每次堆的 **push** 或 **pop** 操作的时间复杂度均为 $O(\log \text{堆大小})$ $O(\log \text{堆大小})$ ，堆大小不超过 $|E||E|$ 。

```
import heapq
from typing import List, Dict, Tuple

def dijkstra(graph: Dict[int, List[Tuple[int, int]]], start: int) ->
    Tuple[List[float], List[int]]:
    """
    堆优化的 Dijkstra 算法，求单源最短路径（非负权图）

    参数:
        graph: 邻接表，格式 {node: [(neighbor, weight), ...]}
        start: 源点编号

    返回:
        dist: 从 start 到每个节点的最短距离列表（下标对应节点编号）
        prev: 前驱节点列表，用于重建路径（-1 表示无前驱）
    """
    n = len(graph)  # 假设节点编号 0 ~ n-1
    INF = float('inf')
    dist = [INF] * n
    prev = [-1] * n
    dist[start] = 0

    # 优先队列存储（当前距离，节点）
    pq = [(0, start)]

    while pq:
        cur_dist, u = heapq.heappop(pq)

        # 如果当前弹出的距离不是该节点的最新记录，则跳过（延迟删除）
        if cur_dist > dist[u]:
            continue

        # 松弛邻居
```

```

    for v, w in graph[u]:
        new_dist = cur_dist + w
        if new_dist < dist[v]:
            dist[v] = new_dist
            prev[v] = u
            heapq.heappush(pq, (new_dist, v))

    return dist, prev

```

4. 包含负权边的一般图：DP的Bellman-Ford 算法 (贝尔曼-福特)

当图中出现了负权边（比如走某条路不仅不消耗油，还倒贴油钱），Dijkstra 失效了，我们需要更暴力的 DP 思路。

- **核心思想**：我不挑顺序了，我直接把图里的所有边遍历一遍，全部尝试松弛。由于一条最长的不带环的最短路径最多包含 $V - 1$ 条边，所以我们把“全边松弛”这个动作重复 $V - 1$ 次，就能保证所有最短路径都被找出来。
- **额外绝技**：如果在第 $V - 1$ 次之后，再跑一次全边松弛，发现还能更新距离，说明图中存在 **负权环 (Negative Cycle)**（在环里无限转圈，距离能趋于无穷小）。因此它常用于检测负权环。
- **时间复杂度**： $O(V \cdot E)$ 。

Bellman-Ford 核心逻辑补充：

1. **$V - 1$ 的数学意义**：在无负权环的图中，最短路径最多包含 $V - 1$ 条边。每轮松弛多覆盖一条边，因此 $V - 1$ 轮必收敛。
2. **负权环的处理**：若第 V 次松弛仍能更新距离，则判定图中存在负权环，此时最短路径问题无解（结果为 $-\infty$ ）。
3. **对比 Dijkstra**：Dijkstra 无法处理负权边是因为它基于贪心策略，一旦确定一个点的最短路就不再更改；而 Bellman-Ford 属于动态规划，通过“全边松弛”多次修正，能够兼容负权边并检测负权环。

```

def bellman_ford(graph, V, E, src):
    """
    graph: 存储边的列表，格式为 [(u, v, w), ...], 表示从 u 到 v 权重为 w
    V: 顶点数量
    E: 边的数量
    src: 起点节点
    """
    # 1. 初始化距离数组，起点设为0，其他设为无穷大
    dist = [float("inf")] * V
    dist[src] = 0

```



```
# 2. 核心松弛步骤: 重复 V-1 次 [cite: 64, 66]
# 因为无负权环图中, 最短路径最多包含 V-1 条边 [cite: 64, 66]
for _ in range(V - 1):
    for u, v, w in graph:
        if dist[u] != float("inf") and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w

# 3. 第 V 次松弛: 检测负权环
# 如果还能更新, 说明存在负权环, 最短路径趋于负无穷
for u, v, w in graph:
    if dist[u] != float("inf") and dist[u] + w < dist[v]:
        print("图中存在负权环! 最短距离在数学上不存在。")
        return None

return dist
```

全源最短路径问题 (All-Pairs Shortest Paths, APSP)

全源最短路径问题 (APSP) 的目标是计算图中任意两个顶点 u 到 v 之间的最短距离。它是单源最短路径 (SSSP) 问题的全局扩展。

1. 核心算法对比表

在处理 APSP 问题时, 算法的选择主要取决于图的 规模 (顶点数 V 、边数 E) 以及 边权特征 (是否包含负权边)。

算法	核心逻辑	适用场景	时间复杂度	空间复杂度	关键评价
Floyd-Warshall	动态规划	稠密图、 小规模图 ($V \leq 500$)	$O(V^3)$	$O(V^2)$	代码极其简洁, 是小规模图的首选。
Johnson	势能重加权 + V 次 Dijkstra	稀疏图、 允许负权边	$O(VE + V^2 \log V)$	$O(V^2 + E)$	解决负权边在稀疏图下效率低下的“神来之笔”。

算法	核心逻辑	适用场景	时间复杂度	空间复杂度	关键评价
多源 Dijkstra	重复 V 次单源算法	仅限非负权边的图	$O(VE + V^2 \log V)$	$O(V^2 + E)$	结构最直观，但无法处理任何负权边。

2. 算法详细拆解

A. Floyd-Warshall 算法（弗洛伊德算法）

这是基于动态规划的经典算法。

- 核心思想**：通过 V 个阶段，依次尝试将每一个节点 k 作为中转点，来更新所有点对 (i, j) 之间的最短路径。
- 状态转移方程**： $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k][j])$ 。
- 空间优化**：原始状态需要 $O(V^3)$ 记录 $dp[k][i][j]$ ，但由于第 k 层的状态只依赖于第 $k-1$ 层，可以压缩到 $O(V^2)$ 。
- 负权环检测**：算法结束后，若发现对角线元素 $dp[i][i] < 0$ ，说明存在经过 i 点的负权环。

B. Johnson 算法

针对稀疏图中 $O(V^3)$ 开销太大的问题，Johnson 算法结合了 Dijkstra 的高效与 Bellman-Ford 的负权处理能力。

- 核心挑战（为什么不能直接加正数）**：如果给所有边统一加一个权重 W ，包含边数多的路径会比边数少的路径增加更多总权重，从而改变“最短”的定义。
- 核心思想（重加权 Reweighting）**：
 - 利用 **Bellman-Ford** 计算每个点的势能 $h(u)$ （顺便检测负权环）。
 - 将边权更新为 $w'(u, v) = w(u, v) + h(u) - h(v)$ 。这能保证所有新边权 $w' \geq 0$ ，且不改变最短路径的拓扑结构。
 - 对每个顶点运行一次 **Dijkstra** 算法。

C. 多源 Dijkstra 算法

在没有任何负权边的情况下，这是最简单高效的方案。

- **执行逻辑：**对图中的每一个节点都作为一次起点，独立调用单源最短路算法。
- **局限性：**由于 Dijkstra 算法基于贪心策略，一旦确定一个点的最短路就不再更改，因此完全无法处理负权边。

3. 算法选型决策流程（笔记建议）

****当你面对全源最短路径问题时，可以按以下逻辑选择算法**

**^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ ^^^ ^^ :

1. 是否有负权边?
 - 否→** 选 **多源 **Dijkstra**^^^^^^^^^^^^^^^^^。
 - 是 → 进入下一步。
2. 是否有负权环?
 - 是→ 无解 **，需通过 Bellman-Ford 或 Floyd 报错退出 **^^^^^^^^^^^^^。
 - 否 → 进入下一步。
3. 图的规模与疏密?
 - 节点少 ($V < 500$), 边稠密→** 选 ****Floyd-Warshall**^^^^^^^^^^^^^^^^^^^^。
 - 节点多, 边稀疏→** 选 ****Johnson**^^^^^^^^^^^^^^^^^^^^。